

Applied Regression Homework 3 Dai Xunlian

225040015

daixunlian

February 2026

Conceptual

5. Comparing LDA and QDA

- (a) **Training:** QDA is more flexible, so its training error is typically no worse than LDA. **Test:** if the true Bayes boundary is linear, LDA is more appropriate and usually generalizes better due to lower variance.
- (b) **Training:** QDA is more flexible and typically fits better. **Test:** if the true boundary is nonlinear and the sample size is large, QDA usually performs better; with small n , LDA may be more stable.
- (c) As n increases, QDA parameter estimates become more stable and variance decreases, so its test accuracy relative to LDA tends to improve (or at least not decline).
- (d) **False.** Even with a linear true boundary, QDA's extra flexibility increases variance and can hurt test performance; LDA is more robust in this case.

6. Logistic regression probability

- (a) Linear predictor: $\eta = -6 + 0.05 \times 40 + 1 \times 3.5 = -0.5$. Probability: $p = \frac{1}{1+e^{-\eta}} = \frac{1}{1+e^{0.5}} \approx 0.3775$.
- (b) Set $p = 0.5$, so $\eta = 0$: $-6 + 0.05x + 3.5 = 0 \Rightarrow 0.05x = 2.5 \Rightarrow x = 50$ hours.

Applied

Data and code

The data come from the ISLR Weekly data set (1990–2010, 1,089 observations). The main Python code is below:

```

import pandas as pd
import numpy as np
import statsmodels.api as sm
from sklearn.metrics import confusion_matrix, accuracy_score
from sklearn.linear_model import LogisticRegression
from sklearn.discriminant_analysis import
    LinearDiscriminantAnalysis, QuadraticDiscriminantAnalysis
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import StandardScaler

url = "https://raw.githubusercontent.com/selva86/datasets/
    master/Weekly.csv"
weekly = pd.read_csv(url)

# (a) Numerical and graphical summaries
summary = weekly.describe()
cor = weekly[["Lag1", "Lag2", "Lag3", "Lag4", "Lag5", "Volume", "
    Today"]].corr()

# (b) Full-data logistic regression
X = weekly[["Lag1", "Lag2", "Lag3", "Lag4", "Lag5", "Volume"]]
y = (weekly["Direction"]=="Up").astype(int)
X_sm = sm.add_constant(X)
logit_full = sm.Logit(y, X_sm).fit()

# (c) Full-data confusion matrix
pred = (logit_full.predict(X_sm) > 0.5).astype(int)
cm_full = confusion_matrix(y, pred)
acc_full = accuracy_score(y, pred)

# (d)-(h) Train 1990-2008, test 2009-2010, Lag2 only
train = weekly["Year"] <= 2008
test = weekly["Year"] >= 2009
Xtr = weekly.loc[train, ["Lag2"]]
Xte = weekly.loc[test, ["Lag2"]]
ytr = y[train]
yte = y[test]

# Logistic
logit = LogisticRegression(solver="liblinear").fit(Xtr, ytr)
cm_logit = confusion_matrix(yte, logit.predict(Xte))
acc_logit = accuracy_score(yte, logit.predict(Xte))

# LDA / QDA
lda = LinearDiscriminantAnalysis().fit(Xtr, ytr)
qda = QuadraticDiscriminantAnalysis().fit(Xtr, ytr)
cm_lda = confusion_matrix(yte, lda.predict(Xte))
cm_qda = confusion_matrix(yte, qda.predict(Xte))

```

```

acc_lda = accuracy_score(yte, lda.predict(Xte))
acc_qda = accuracy_score(yte, qda.predict(Xte))

# KNN (k=1)
knn = KNeighborsClassifier(n_neighbors=1).fit(Xtr, ytr)
cm_knn = confusion_matrix(yte, knn.predict(Xte))
acc_knn = accuracy_score(yte, knn.predict(Xte))

# Naive Bayes
nb = GaussianNB().fit(Xtr, ytr)
cm_nb = confusion_matrix(yte, nb.predict(Xte))
acc_nb = accuracy_score(yte, nb.predict(Xte))

```

13(a) Numerical and graphical summaries

Numerical summary: Up weeks = 605, Down weeks = 484; linear correlations between lagged returns and Today are weak (e.g., $\text{corr}(\text{Lag2}, \text{Today}) \approx 0.059$).

Graphical summary: see Figures 1–3.

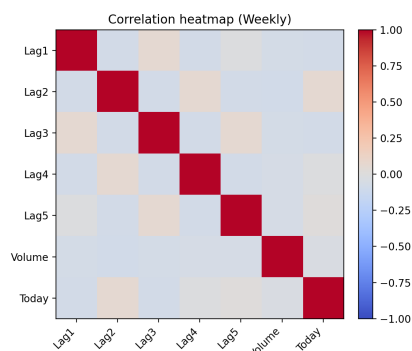


Figure 1: Correlation heatmap of Lag/Volume/Today

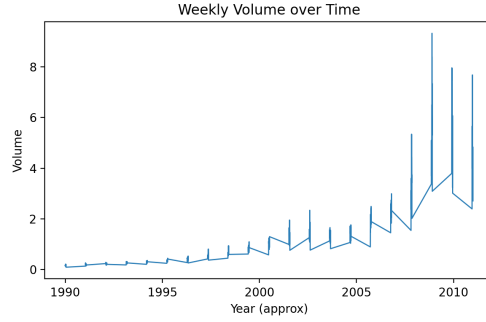


Figure 2: Volume over time (overall upward trend)

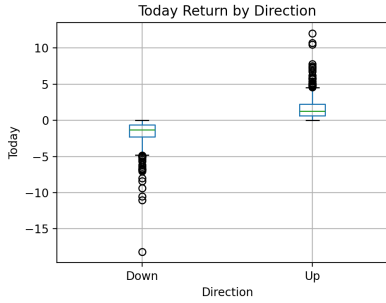


Figure 3: Distribution of Today by Direction

Brief conclusion: lagged returns are weakly correlated with Today; Volume generally rises over time; the Up proportion is slightly higher than Down.

13(b) Full-data logistic regression significance

The logistic regression shows **Lag2** is significant at the 5% level ($p \approx 0.03$); the other predictors are not significant. This provides weak evidence that the second lag is associated with direction.

13(c) Full-data confusion matrix

	Pred Down	Pred Up
Actual Down	54	430
Actual Up	48	557

Table 1: Full-data logistic regression confusion matrix (threshold 0.5), accuracy ≈ 0.561

Interpretation: the model tends to predict Up, resulting in many Down weeks misclassified as Up (FP=430). Overall accuracy is about 56%.

13(d)–(h) Train 1990–2008, test 2009–2010 (Lag2 only)

Method	TN	FP	FN	TP
Logistic	9	34	5	56
LDA	9	34	5	56
QDA	0	43	0	61
KNN (k=1)	21	22	30	31
Naive Bayes	0	43	0	61

Table 2: Test-set confusion matrices (2009–2010); accuracies are 0.625, 0.625, 0.5865, 0.500, 0.5865

13(i) Best method (based on 13(d)–(h))

Under this setup, **Logistic and LDA tie for best** (test accuracy 0.625).

13(j) Exploring predictor combinations and transformations

After trying combinations, squared terms, interactions, and multiple KNN k values, the best result is:

KNN (k=5) with predictors: Lag2 and Lag3², test accuracy ≈ 0.6827 , confusion matrix:

	Pred Down	Pred Up
Actual Down	25	18
Actual Up	15	46

Table 3: Best-performing combination (test set)

15. Functions (Python version)

(a)–(d) Power functions and values

```
# (a) Power(): 2^3
def Power():
    print(2**3)

# (b) Power2(x, a): print x^a
def Power2(x, a):
    print(x**a)
```

```

# (d) Power3(x, a): return x^a
def Power3(x, a):
    result = x**a
    return result

# (c) Compute
print(Power3(10, 3))    # 1000
print(Power3(8, 17))    # 2251799813685248
print(Power3(13, 13))   # 302875106592253

```

Computed values: $10^3 = 1000$, $8^{17} = 2251799813685248$, $13^{13} = 302875106592253$.

(e) Plot of $f(x) = x^2$

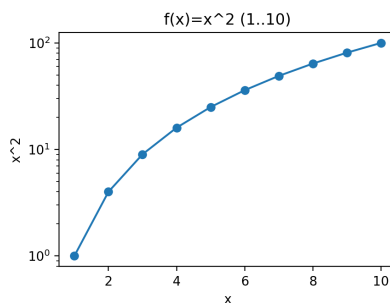


Figure 4: $f(x) = x^2$ ($x = 1..10$, y-axis on log scale)

(f) PlotPower() function

```

import numpy as np
import matplotlib.pyplot as plt

def PlotPower(x, a):
    x = np.asarray(x)
    y = x**a
    plt.plot(x, y, marker='o')
    plt.xlabel('x')
    plt.ylabel(f'x^{a}')
    plt.title(f'PlotPower(x, a={a})')
    plt.show()

# Example
PlotPower(range(1, 11), 3)

```

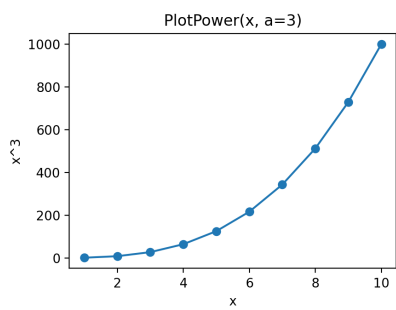


Figure 5: Example: PlotPower(1..10, 3)